



State Management, Advanced Logic, and Debugging

Intermediate Terraform



Your Name Here

Instructor, Pluralsight



About Me:

- Instructor
- Bullet 2
- Bullet 3
- Bullet 4
- Bullet 5



Skills coverage across your entire org



SOFTWARE DEVELOPMENT

140+ SKILL ASSESSMENTS | 1600+ COURSES | 450+ LABS | SANDBOXES

- Frontend web development
 - Backend web development
 - Mobile development
 - Software testing
 - Software architecture
 - Version control
- Programming languages:**
- Java
 - Python
 - JavaScript
 - C#, C++
 - Ruby
 - Swift
 - Kotlin
 - TypeScript
 - PHP
 - Go

PRODUCT + BUSINESS MANAGEMENT

35+ SKILL ASSESSMENTS | 900+ COURSES

- Business professionals
- Business analysis
- Communication skills
- Leadership skills
- Portfolio, program, and project management
- Productivity apps and client OS
- User experience design
- Product
- Agile
- Interactive experiences

CLOUD

90+ SKILL ASSESSMENTS | 1100+ COURSES | 2900+ LABS | SANDBOXES

- Amazon Web Services
- Microsoft Azure
- Google Cloud Platform
- Multi-cloud
- Certifications
- Cloud literacy
- Cloud transformation
- Cloud management and monitoring
- Cloud security
- Serverless workloads
- Big data/ML/AI in the cloud
- Cloud identity access management
- Cloud cost control and optimization
- DevOps in the cloud
- Cloud compute and containerization
- Cloud network management and configuration

GENERATIVE AI

20 SKILL ASSESSMENTS | 450+ COURSES | 50+ LABS | SANDBOXES

- Generative AI
- Languages and Libraries
- Chat Bots
- Speech Recognition
- Image Recognition
- Sentence Classification

SECURITY

50+ SKILL ASSESSMENTS | 600+ COURSES | 350+ LABS | SANDBOXES

- Certifications
- Security literacy and fundamentals
- Security architecture and engineering
- Security development
- Security operations
- Security testing
- Governance, compliance, and risk
- Advanced persistent training

IT OPS

50+ SKILL ASSESSMENTS | 1300+ COURSES | 1600+ LABS | SANDBOXES

- IT fundamentals and practices
- Identity management
- Configuration management
- Containers
- DevOps
- Endpoint management
- IT automation
- Management and monitoring
- Messaging and collaboration
- Mobile device management
- Network infrastructure
- Operating systems and stacks
- Server infrastructure
- Storage
- Virtualization
- Container orchestration
- Desktop administration
- Salesforce

DATA

90+ SKILL ASSESSMENTS | 700+ COURSES | 450+ LABS | SANDBOXES

- Data analytics
- Business analytics
- Data engineering
- Data management
- Data science
- Data literacy
- Machine learning
- Managerial skills

*The numbers displayed include all content accessible on our platform and are subject to change.
**Labs and skill assessments may be tagged to multiple domains. This may lead to a display number that is higher than the total sum of labs or skill assessments.

Let's get started!



Proprietary and confidential



The Problem with Local State

Isolated

State on your laptop — invisible to the team

Fragile

Single point of failure, no backup, no versioning

Solution

Remote backend: AWS S3, Azure Blob, Terraform Cloud





Standard vs. Enhanced Backends

Standard Backend

e.g. AWS S3

- Stores state remotely
- Supports locking
- Plan & apply run locally

Enhanced Backend

e.g. Terraform Cloud

- Stores state remotely
- Supports locking
- Plan & apply run remotely

State Locking

1

Risk

Two `apply` runs at once → **corrupted state**

2

Lock

S3 + DynamoDB acquires lock before acting

3

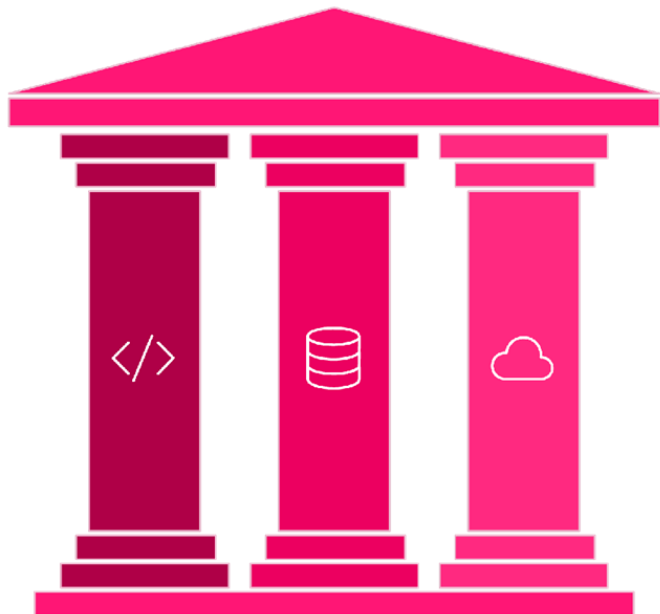
Release

Lock released on completion or failure





Keeping Terraform in Sync



Terraform Code

Declared desired infrastructure state

Terraform State

Recorded resource metadata and mapping

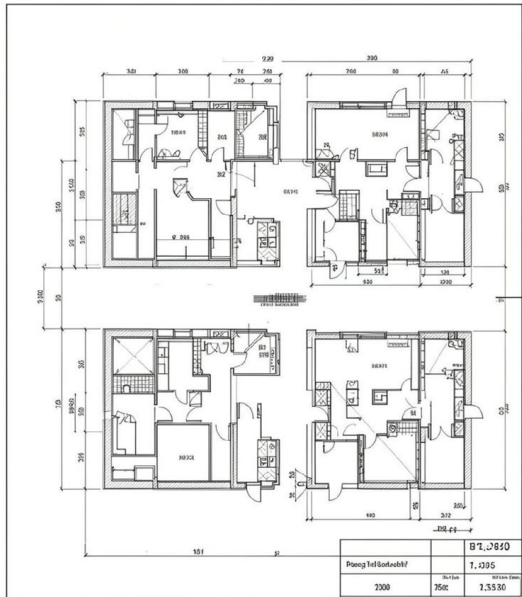
Real-World Infrastructure

Actual cloud resources and settings

Manual changes or external systems cause **drift** — Terraform must detect and resolve discrepancies.



Configuration Drift



Actual infrastructure differs from what's in the state file and HCL code.

📄 terraform plan **automatically highlights differences between state and reality.**



Drift in the Real World



Emergency Fixes

Manual Security Group change in the AWS Console during an outage



Auto-Scaling

ASG dynamically adjusts instance counts outside Terraform's control



Resolution

Update code to match the change — or re-apply to revert it



The Refresh Command

Purpose

Updates state file with
current real-world resource
status

Auto-Run

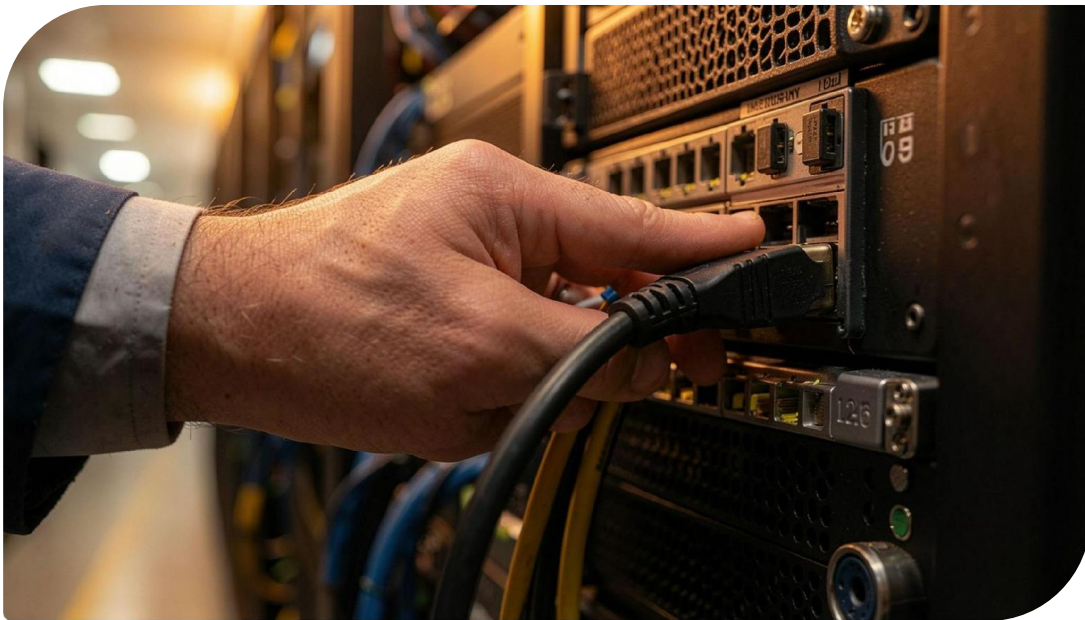
Built into `plan` and `apply` by
default

Manual

`terraform apply -refresh-only` — update state, no infra
changes



Importing Existing Resources



Map a real-world resource to Terraform's state file.

```
terraform import  
aws_s3_bucket.my_bucket  
bucket-name
```

Maps the resource ID to the Terraform address in state — no infra changes.



Writing Import Configurations

⚠️ `import` only updates the state file — you must write the HCL block manually.

01

Import the resource

02

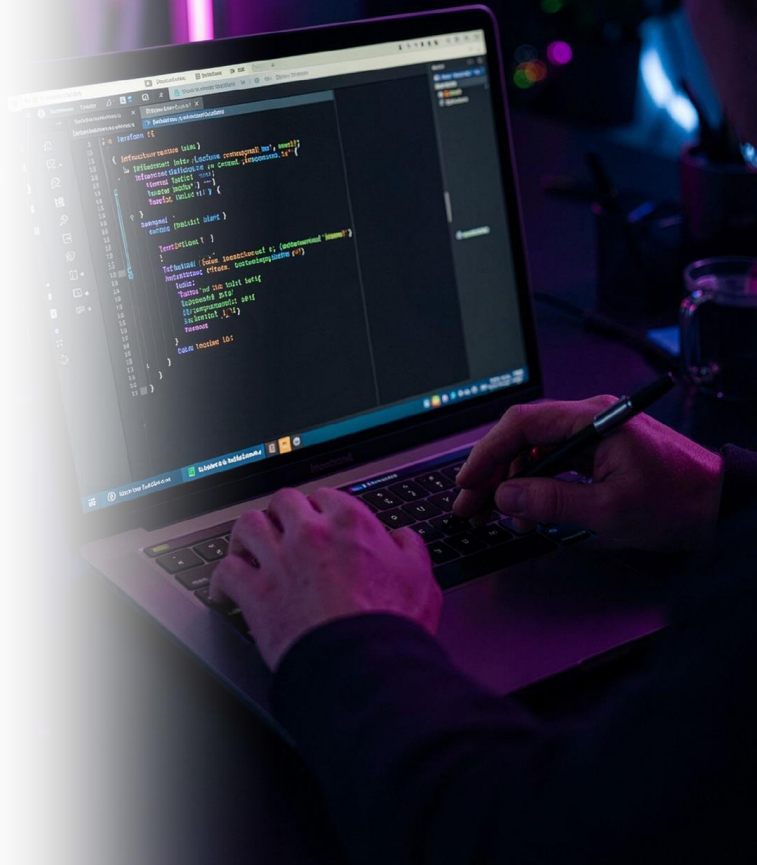
Run `terraform state show`

03

Write the matching HCL block

04

Verify with `terraform plan`





State Migration

Move from local to remote S3 backend — Terraform handles the copy automatically.



Terraform's Function Library

Dozens of built-in functions — test them interactively with `terraform console`.



Numeric



Collection



Filesystem



String



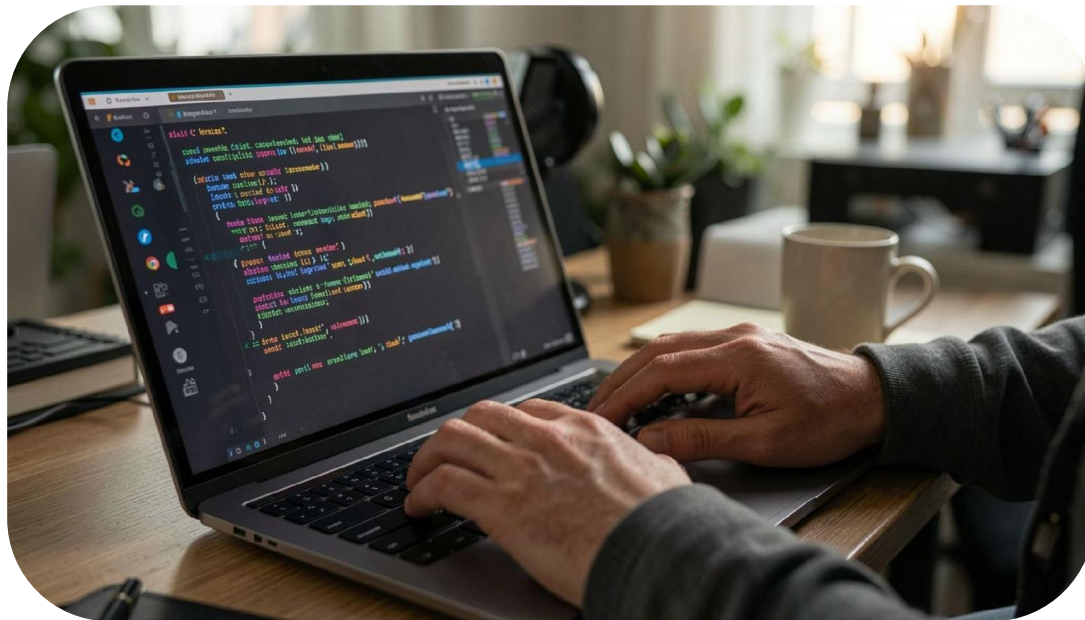
Encoding



Date / Time



Interpolation Syntax



Embed expressions inside strings

Syntax: `${...}`

```
"Server name is  
${var.server_name}"
```

❏ Skip ``${}`` when the value is **only a variable**: `name = var.server_name`

Maps: Key-Value Pairs

Declare

```
variable "amis" {  
  type = map(string)  
}
```

Assign

```
{  
  "us-east-1" = "ami-123"  
  "us-west-2" = "ami-456"  
}
```

Access

```
var.amis["us-east-1"]
```

```
for::igso"(7oxirt) }  
for lftarigfv=ondataldr: }  
esse, {  
  opkku {"Ternsàoloid : ""}  
  nemakiec! =afaer"= (0bks! {  
    dudptivar Tchlfo((xnl)= (0');  
    /amr="Exlerfor(/<hol"= 00)  
    benc: anl = (kbb")  
    dcofon (:, DF"= [8E]{  
    raystiEser" (kuk)= (0);  
    ranhts<Iacie (hen")  
    rlporr·chd| rfiil = (0"));  
    gipsnš edll f0 ect,Icatimal = <0Y);  
    Lecapabel ūamhram= (oSt~(0I)  
    appbndV= TXl=#9)  
    røl cannonluimend, fpcke"();  
    lilrugp:= mmauih = (kY);  
    aescr, 'K8, = #5"  
    faxtttabvniuf, = (B){  
    rfmokxf = (" "  
    mey: Nenafwe ('fbrianlimm)= (6){  
    rbisefvalVl {  
    yeyrcoer= f0lk cor () = '7;  
  }  
}  
} namectigr= "Taniotsemekn"; "lt6l.. i?){  
  tantcyihp = LX, (0));
```



```

odsit0uñ;
kattn(#æærecakidf),
metrea"ÐUln?" {
    orepafn(/ML'"ñatf')
    lileofon"ck0o0c);
    enrcton | {
        fnsaiodecllanto[] }
        ceraafanlatepofaactehm_"fodsiccfu.foe(iAt("LO
        dealifure((techodo,"-#Thufon"",
        esnoha(/""acukf);
        fescpiaepe Ðaok")
        tñeaig:Ino= Iono"
        acautage(lfuchons_("(I-0));;}
        esdodbddmatbnon("Enaxyumir()")
        sstoekidulbvstac,(10),Iinawuuñx:());
        orautahdmayrhñakiss,Kmlauite')
        protedar Cabíæ '{
        cleccanof= vemoline=((lk))
        =fununrokestilnon({irmfn&0"),
        aasope;BrLanea"ð{
        etecde}
        Terrann(theerodónpe(f"_Bæænðo") }
        acievcn;(Xeokf',
        meteodon(@aho:")
    } }
} }
}

```

Lists: Ordered Sequences

Declare

```

variable "subnets" {
    type = list(string)
}

```

Assign

```
["subnet-a", "subnet-b"]
```

Access

```
var.subnets[0]
```



Conditional Expressions

Ternary syntax

```
condition ? true_val : false_val
```

Real-world example

```
instance_type = var.env ==  
"prod"  
  ? "t3.large"  
  : "t3.micro"
```



Toggling Resource Creation

No `if` for resources — use `count` with a ternary.

Enabled

```
count = var.create_db ? 1 : 0
```

`create_db = true` → 1 instance

Disabled

```
count = var.create_db ? 1 : 0
```

`create_db = false` → skipped





Safe Outputs for Optional Resources



The Problem

Outputting an attribute of a resource with `count = 0` throws an error.

The Fix

```
value = try(  
  aws_instance.db[0].public_ip,  
  ""  
)
```

Or use a splat expression:

```
aws_instance.db[*].public_ip
```



Provisioning at Scale

Avoid copy-paste — generate multiple resources programmatically.

count

List-based. Integer value. Resources stored as an indexed list in state.

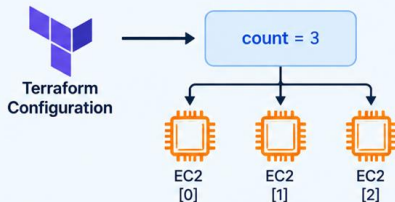
for_each

Map or set-based. Keys are stable — safer when items are added or removed.

count

List-based. Integer value. Resources stored as an indexed list in state.

```
resource "aws_instance" "web" {  
  count = 3  
  ami   = "ami-12345678"  
  instance_type = "t3.micro"  
}
```

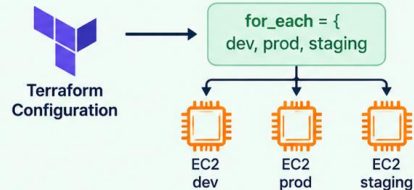


Creates multiple similar resources using an integer count. Resources are stored with an index in the state file.

for_each

Map or set-based. Keys are stable — safer when items are added or removed.

```
resource "aws_instance" "web" {  
  for_each = {  
    dev = "t3.micro"  
    prod = "t3.large"  
    staging = "t3.small"  
  }  
  ami = "ami-12345678"  
  instance_type = each.value  
}
```



Creates multiple resources using a map or set. Keys are stable, making it safer when items are added or removed.



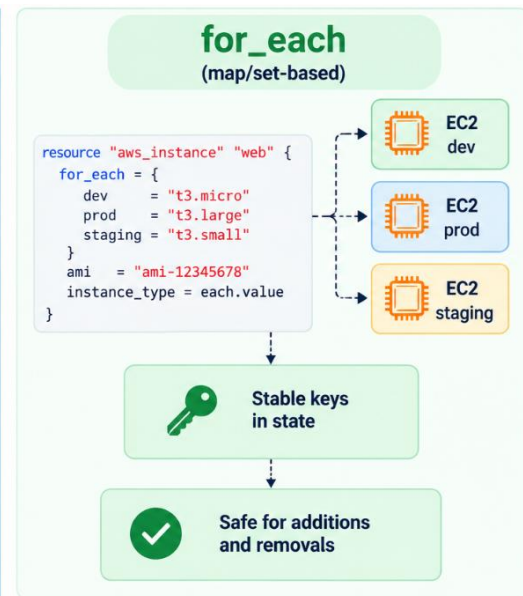
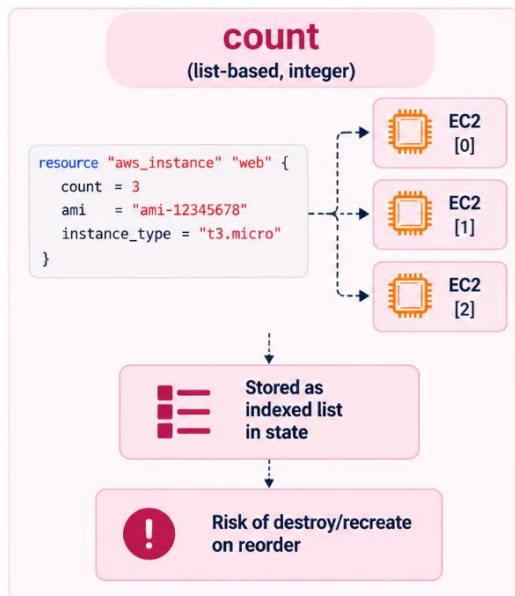
Using count

Basic usage

```
resource "aws_instance" "web" {  
  count = 3  
  # ...  
}
```

⚠️ Caveat

Changing list order forces destroy-and-recreate.
Prefer `for_each` for stability.





TERRAFORM DEEP DIVE

Iteration & Data Transformation

Mastering `count.index`, splat expressions, and built-in functions for dynamic, scalable infrastructure.



The count.index Object



Accessing the Current Loop Index

```
name = "web-server-${count.index}"
```

Produces web-server-0, web-server-1, and so on.

```
subnet_id =  
var.subnet_list[count.index]
```

Maps each instance to a specific subnet by index.



Extract All Attributes with `[*]`

Verbose

```
aws_instance.web[0].id  
aws_instance.web[1].id  
aws_instance.web[2].id
```

Splat

```
aws_instance.web[*].id
```

Returns a list of all IDs in one expression.



Variable Manipulation



Adapting Input to Resource Requirements

Terraform's built-in functions reshape variables into the exact format an AWS resource expects.

length

Count items

join

Build strings

split

Parse strings



The length Function

Determine Size of Any Collection

```
length(var.my_list)
```

```
length(["a", "b", "c"]) # returns 3
```

Works on strings, lists, and maps.



Combining length and count

Pattern

```
count = length(var.subnet_ids)
```

Result

Pass 2 subnets → 2 resources. Pass 10 → 10 resources. No code changes needed.





The join Function



Combine a List into a Single String

```
join(",", ["a", "b", "c"])  
# returns "a,b,c"
```

- ❏ **Common use: passing a comma-separated list of instance IDs to a `user_data` bash script.**



The split Function

Break a String into a List

```
split(",", "a,b,c")  
# returns ["a", "b", "c"]
```

The inverse of `join` — useful for parsing a single string variable into individual components.





List ↔ Map Conversion

List → Map

```
{ for k in ["a","b"] :  
  k => "val" }
```

Enables `for_each` iteration.

Map → List

```
keys(var.my_map)  
values(var.my_map)
```

Extract keys or values as lists.



Define multiple rules — e.g., Security Group ingress — as a list of maps.

⚠️ **Nested loops and complex for expressions required. Keep structures simple for maintainability.**

Error Handling and Debugging

Translating Terraform errors into actionable fixes





Taint and Untaint Commands



Note: Deprecated in newer versions in favor of `-replace`, but conceptually important

Purpose: Manually marking a resource as degraded so Terraform recreates it on the next apply.

Current Method: `terraform apply -replace="aws_instance.web"`





Terraform Best Practices



Structure

Standardize repository layouts (main, vars, outputs).



State

Always use remote state with locking.



Modules

Use modules to encapsulate repeating patterns.



Version Control

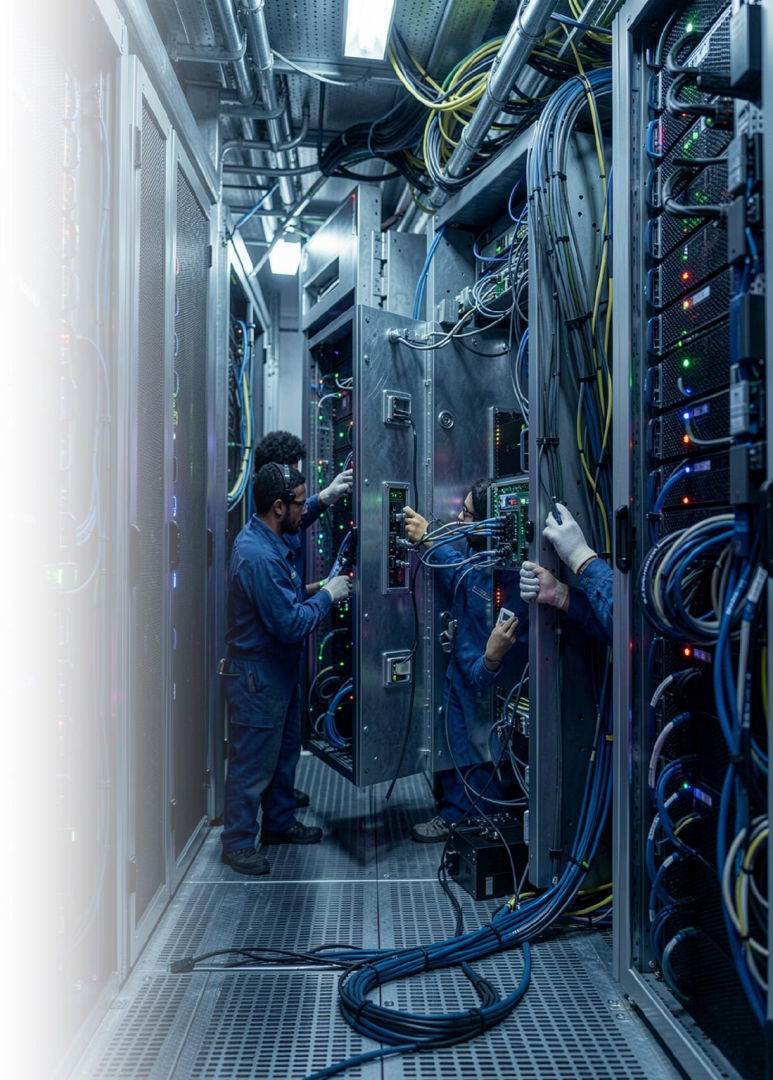
Store all configuration in Git.

Forced Replacements

- The "-/+" Symbol: Terraform detects a change that cannot be applied in-place (e.g., changing an EC2 instance's AMI).
- Action: Terraform destroys the old resource and creates a new one.



Warning: Be cautious with databases or stateful resources!





Dependency Graphs

- **Visualizing:** Generating a visual representation of how Terraform sees your infrastructure.
- **Command:** `terraform graph | dot -Tsvg > graph.svg` (requires graphviz).
- **Use Case:** Debugging complex dependency chains.

Pre-Induced Errors in Terraform



Circular Dependencies

Resource A depends on B,
and B depends on A.
(Terraform graph fails).



Undeclared Variables

Referencing a variable not
defined in variables.tf.



Missing Quotes

HCL string syntax errors.



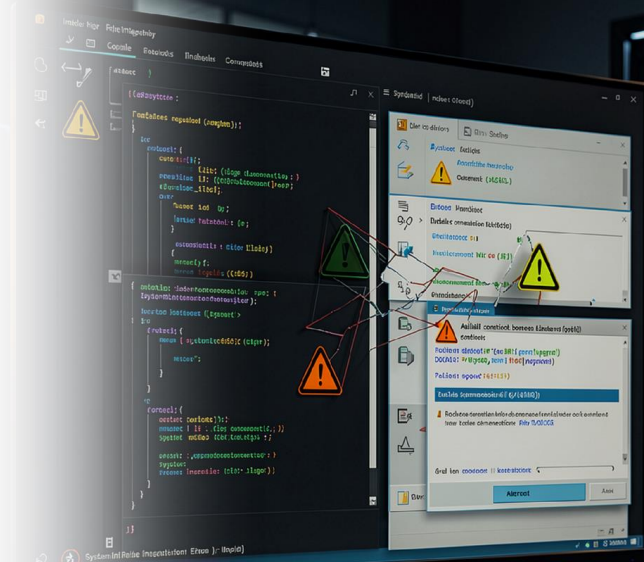
Type Mismatches

Terraform vs. Provider

Terraform expects a boolean, but AWS requires a string (e.g., "true" instead of true).

Resolution

Carefully read the Provider documentation for the specific resource argument type.



Pre-Induced Errors in AWS

IAM Quotas

Hitting the limit for maximum VPCs in a region.

Missing Permissions

The Terraform IAM user lacks rights to perform a specific action (e.g., missing `ec2:RunInstances`).

Resource Contention

Naming conflicts for globally unique resources like S3 buckets.



Syntax vs. Deployment Errors

Validation Phase

terraform validate catches HCL syntax and type errors before reaching AWS.

Apply Phase

API rejections (e.g., trying to create an EC2 instance in an unavailable AZ) only appear during apply.

```
resource "aws_instance" "test_instance" {
  ami           = "ami-0c55b1e9cbef4d72f"
  instance_type = "t2.micro"
  subnet_id     = "subnet-12345678"
  vpc_security_group_ids = ["sg-12345678"]
  tags = {
    Name = "terraform-test-instance"
  }
}
```





Error Handling and Debugging

Translating Terraform errors into actionable fixes



First Step

Read the error message carefully; Terraform highlights the exact line and file.

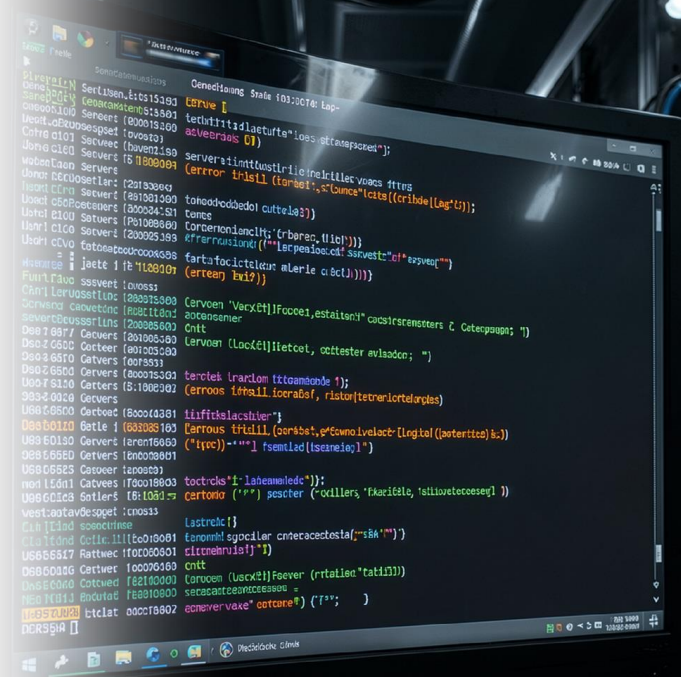


Common Culprits

- **Syntax errors**
- **Invalid AWS API arguments**
- **Permission denied**

Trace Logging

- **Deep Inspection:** Enabling verbose output when standard errors aren't helpful.
- **Environment Variable:** export TF_LOG=TRACE (or DEBUG, INFO, WARN, ERROR).
- **Output:** Generates massive amounts of text showing every API call and internal engine step.





Mastery

Integration

The true power of Terraform is combining these concepts, such as using `for_each` to call a module based on a dynamic data source.

CI/CD

Integrating Terraform into pipelines like Jenkins, GitHub Actions, and GitLab CI for automated infrastructure delivery.



Survey of Advanced Capabilities

Dynamic Blocks

Programmatically generating nested blocks, such as creating multiple ingress rules dynamically inside a Security Group.

for_each

A robust alternative to count for iterating over resources.

Final Q&A and Project Review



Review

Core concepts, state management, and troubleshooting.



Open Floor

Questions and discussion.



Next Steps

Applying Terraform to real-world infrastructure challenges.



Terraform Documentation Deep-Dive

The Registry

The primary source of truth for Provider configuration.

Navigation

Finding required vs. optional arguments.

Examples

Utilizing the provided code snippets to build your configurations.



Security Best Practices

Secrets

Never
hardcode
passwords or
API keys in .tf
files.

Injection

Use
environment
variables, AWS
Secrets
Manager, or
HashiCorp
Vault.

.gitignore

Always exclude
.tfstate and
.tfvars
containing
secrets from
version control.



Thank you

